

TP java

Matisse
kra
bts sio 1



Sommaire

Problème :

Sous problème 1 : résolution de quizz

Rappel des règles principales

Rappel règles pour identifiants de méthodes (Java) :

Sous problème 2 : écrire et compiler

programme 1 : chanson préférer

programme 2 : chaise et table

programme 3 : le triangle

Sous problème 3 : debug

Sous problème 4 : création d'un jeu

Sous problème 5 : écrire des programmes pour le restaurant yummy

conclusion

Annexe

Problème :

l'objectif est d' apprendre à maîtriser le langage java a travers plusieurs sous problèmes et cas pratiques.

travail effectuer sur mac sous les logiciel eclipse et vs code

Sous problème 1 : résolution de quizz

j'ai résolu un premier quizz portant sur des connaissance du java :

voici les réponse de ce premier quizz

1. Le langage machine le plus basique niveau circuit est :
 - a. Le langage machine
2. Les langages qui permettent d'utiliser un vocabulaire qui utilise les termes : read, write ou add sont :
 - b. Haut niveau
3. Les règles du langage de programmation constituent :
 - a. la syntaxe
4. Un _____ traduit les instructions de langage de haut niveau en code machine
 - c. un compilateur
5. Les emplacements de mémoire nommés de l'ordinateur sont appelés :
 - b. variables
6. Les opérations individuelles utilisées dans un programme informatique sont souvent regroupées en unités logiques appelées :
 - a. procédures
7. Une instance de classe est :
 - c. un objet
8. Java a une architecture :
 - a. neutre
9. Vous devez compiler les classes écrites en Java dans :
 - a. un bytecode

10. Toutes les instructions de programmation Java doivent se terminer par :

c. un point-virgule

j'ai résolu un deuxième quizz cette fois portant sur la légalité des identifiant de classe

Rappel des règles principales

1. **Légal** : un identifiant peut contenir lettres, chiffres, _ et \$, mais ne peut pas commencer par un chiffre, ni être un mot-clé réservé.
2. **Conventionnel** :
 - Les noms de classes commencent par une majuscule Pas de caractères spéciaux (sauf _ exceptionnellement, mais peu recommandé).
 - Éviter les abréviations ou notations non expressives.

réponse :

- a. maClasse : **Légal** mais pas conventionnel
- b. void : **Illégal**, car mot-clé réservé.
- c. Golden Retriever : **Illégal**, car espace interdit.
- d. invoice# : **Illégal**, car # interdit.
- e. 36535CodePostal : **Illégal**, car commence par un chiffre.
- f. Appartement : **Légal et conventionnel**.
- g. Fruit : **Légal et conventionnel**.
- h. 8888 : **Illégal**, car uniquement des chiffres.
- i. EcranTotal() : **Illégal**, car () n'est pas permis dans un identifiant.
- j. Acompte_recevable : **Légal** mais pas conventionnel

j'ai résolu un troisième quizz cette fois sur la légalité des identifiant méthode

Rappel règles pour identifiants de méthodes (Java) :

1. **Syntaxe légale** :
 - Peut contenir lettres, chiffres, _, \$.
 - Ne peut pas commencer par un chiffre.
 - Ne doit pas être un mot-clé réservé.
 - Pas d'espace ni de caractère spécial interdit (#, (), etc. — sauf () qui est la déclaration d'appel, pas dans le nom).
2. **Convention de style** :

- Méthodes : donc commencent par une minuscule.
- Nom doit être un verbe ou exprimer une action (getValue, printReport, etc.).
- Pas d'underscore en général (sauf constants ou cas particuliers).

Réponse :

- a. associationRoles() : *Légal et conventionnel* .
- b. void() : *Illégal, car void est un mot-clé réservé.*
- c. Golden Retriever() : *Illégal, car espace interdit.*
- d. invoice#() : *Illégal, car # n'est pas autorisé.*
- e. 24500CodePostal() : *Illégal, car commence par un chiffre.*
- f. PayrollApp() : *Légal mais pas conventionnel*
- g. getReady() : *Légal et conventionnel*
- h. 911() : *Illégal, car ne peut pas être uniquement numérique.*
- i. EcranTotal() : *Légal mais pas conventionnel*
- j. Acompte_Recevable() : *Légal mais pas conventionnel* .

Sous problème 2 : écrire et compiler

j'ai écrit plusieurs programmes différents puis je les ai compilés. Ces exercices permettent de mettre en pratique ce que l'on a vu précédemment :

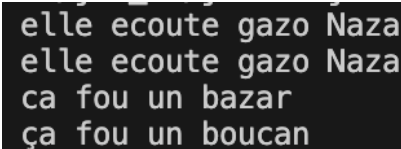
programme 1 : chanson préférée

je devais écrire et compiler un code pour afficher quatre lignes des paroles de ma chanson préférée.

- j'ai utilisé une classe qui utilise l'instruction `println()` pour afficher les paroles de ma musique préférée "connecter de gazo et naza" [connecter de gazo et naza](#)

résultat :

```
1 package parole_chanson;
2
3 import java.util.Scanner;
4
5 public class Parole_chanson {
6
7     public static void main(String[] args) {
8         // TODO Auto-generated method stub
9         System.out.println("elle ecoute gazo Naza");
10        System.out.println("elle ecoute gazo Naza");
11        System.out.println("ca fou un bazar");
12        System.out.println("ca fou un boucan");
13    }
14 }
15
16
17
```



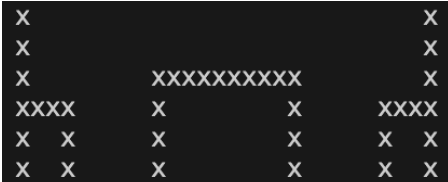
programme 2 : chaise et table

cette fois-ci je devais dessiner une chaise et une table avec des croix

- j'ai utilisé la même méthode que pour la chanson

résultat :

```
1 package parole_chanson;
2
3 public class TableetChaise {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7         System.out.println("x");
8         System.out.println("x");
9         System.out.println("x      xxxxxxxxxx      x");
10        System.out.println("xxxx  x      x  xxxxx");
11        System.out.println("x  x  x      x  x  x");
12        System.out.println("x  x  x      x  x  x");
13    }
14 }
15
16
17
```



programme 3 : le triangle

Pour ce programme j'ai une méthode différente j'ai vu que le symbole se répète dans un ordre favorisant l'utilisation de la boucle For j'ai donc opté pour cette solution.

- j'ai initialisé mon programme pour que cela s'arrête à 7 ligne de T
- j'ai ajouté des espaces au début pour centrer la ligne. Plus on descend (i augmente), moins il y a d'espaces.

Exemple si lignes = 7 :

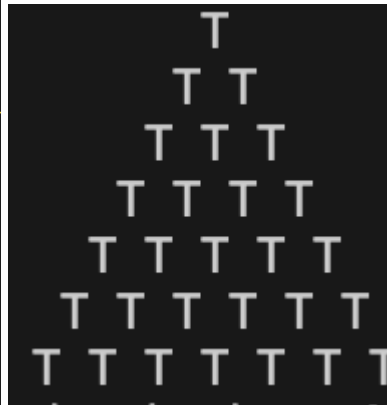
- $i = 1 \rightarrow 6$ espaces
- $i = 2 \rightarrow 5$ espaces
- $i = 3 \rightarrow 4$ espaces
- Après avoir affiché une ligne complète, on saute à la ligne suivante. Sans ça, tout s'afficherait sur une seule ligne.

résultat :

```

1 package triangle;
2
3 public class Algo_Triangle {
4
5     Run | Debug
6     public static void main(String[] args) {
7         // TODO Auto-generated method stub
8         int lignes = 7; // nombre de lignes de ton sapin
9
10        for (int i = 1; i <= lignes; i++) {
11            // Afficher les espaces pour centrer
12            for (int espace = 0; espace < lignes - i; espace++) {
13                System.out.print(s: " ");
14            }
15            // Afficher les T (2*i - 1 pour avoir 1, 3, 5, 7...)
16            for (int t = 0; t < i; t++) {
17                System.out.print(s: "T ");
18            }
19            System.out.println(); // passer à la ligne
20        }
21    }
22 }
23
24
25

```



Sous probleme 3 : debug

exercice 1

Dans ce code, il manque une accolade { après Debug1. Il faut ajouter des points entre System et out, ainsi qu'entre out et println. Ensuite, il faut ajouter un n à println pour obtenir println. Il manque également un guillemet après "Salut". Il faut remplacer le point par un point-virgule, et enfin ajouter une accolade } à la fin du code.

résultat :

code avec erreur

```
1 public class Debug1
2
3     /* This program displays a greeting */
4     public static void main(String[] args)
5     {
6         Systemoutprintln("Salut").
7     }
```

code sans erreur

```
1 public class Debug1 {
2
3     /* This program displays a greeting */
4     public static void main(String[] args)
5     {
6         System.out.println("Salut");
7     }
8 }
```

exercice 2

Ici, il faut d'abord ajouter `*/` pour terminer le commentaire en anglais. Ensuite, il faut ajouter `[` après `String`. Il faut également remplacer le `è` de `être` par un `e` afin d'éviter une erreur. Enfin, il faut corriger la faute où il est écrit `prnitln` et la remplacer par `println`.

resultat :

code avec erreur

```
1 public class Debug2
2 {
3     /* This program displays some output
4     public static void main(String args)
5     {
6         System.out.println("Programmer en java est fun.");
7         System.out.println("Faire un programme");
8         System.out.println("peut être un challenge,");
9         System.out.prnitln("mais quand la syntaxe est correcte,");
10        System.out.println("c'est satisfaisant");
11    }
12 }
```

code sans erreur

```
1 public class Debug2
2 {
3     /* This program displays some output*/
4     Run | Debug
5     public static void main(String args [])
6     {
7         System.out.println(x:"Programmer en java est fun.");
8         System.out.println(x:"Faire un programme");
9         System.out.println(x:"peut etre un challenge,");
10        System.out.println(x:"mais quand la syntaxe est correcte,");
11        System.out.println(x:"c'est satisfaisant");
12    }
```

exercice 3

Dans ce code, il faut remplacer `Debug33` par `Debug3` pour assurer la cohérence entre la classe et le projet. Ensuite, il faut mettre une majuscule au deuxième `System`. Tous les `out` doivent être minuscule. Il faut également remplacer le `T` majuscule du troisième `System` par un `t` minuscule. Enfin, toutes les lettres accentuées dans les `println` doivent être remplacées par leur équivalent sans accent (par exemple, `è` → `e`, `à` → `a`).

resultat :

code avec erreur

```
1 public class Debug33
2 {
3     Run | Debug
4     public static void main(String[] args)
5     {
6         System.Out.println("Derrière la rivière");
7         system.out.println("et au dela du bois");
8         SysTem.Out.println("à la maison du garde nous irons");
9     }
10 }
```


code sans faute

```
1 public class Debug3
2 {
3     Run | Debug
4     public static void main(String[] args)
5     {
6         System.out.println(x:"Derriere la riviere");
7         System.out.println(x:"et au dela du bois");
8         System.out.println(x:"a la maison du garde nous irons");
9     }
}
```

exercice 4

Ajouter le type de retour void dans la méthode main, sinon le programme ne se compile pas.→ On écrit `public static void main(String[] args)` au lieu de `public static main(String[] args)`.

Mettre le texte du message dans des guillemets pour que ce soit une chaîne de caractères valide.→ Il faut écrire "1er GUI program" avec des guillemets autour, sinon Java ne le reconnaît pas comme une phrase.

Ajouter un point-virgule à la fin de l'instruction car toutes les instructions Java doivent se terminer par ;→ On écrit `JOptionPane.showMessageDialog(null, "1er GUI program");` Supprimer le point d'exclamation après l'appel de méthode, car il n'a rien à faire là.→ Il faut enlever le ! à la fin.

reslutat :

code avec erreur

```
1 import javax.swing.JOptionPane;
2 public class Debug4
3 {
4     Run | Debug
5     public static main(String[] args)
6     {
7         JOptionPane.showMessageDialog(null, 1er GUI program)!
8     }
}
```

code sans erreur

```
1 import javax.swing.JOptionPane;
2
3 public class Debug4 {
4     Run | Debug
5     public static void main(String[] args) {
6         JOptionPane.showMessageDialog(parentComponent:null, message:"1er GUI program");
7     }
}
```

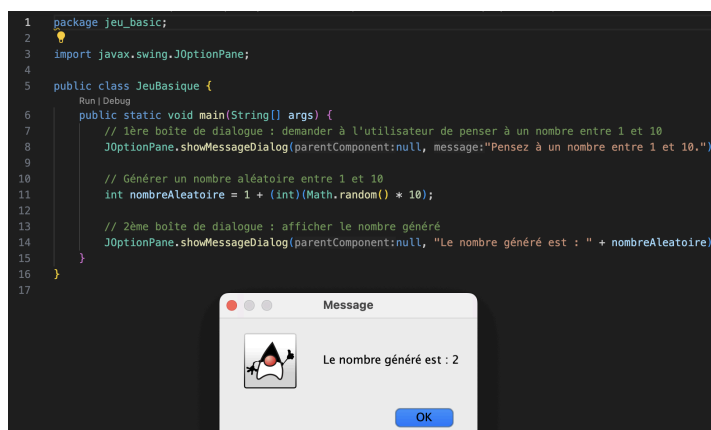
resultat du code :



Sous problème 4 : creation d'un jeu

- j'ai Affiché une première boîte de dialogue qui demande à l'utilisateur de penser à un nombre entre 1 et 10
- j'ai ensuite fait afficher une seconde boîte de dialogue qui affiche un nombre aléatoire entre 1 et 10. L'utilisateur pourra alors voir si le nombre généré correspond à celui qu'il avait en tête.
- j'ai donc utiliser une class qui utilise l'instruction ci-dessou pour générer et utiliser une boite de dialogue affichant un nombre aléatoire compris entre 1 et 10 :

```
JOptionPane.showMessageDialog(null,"le nombre est "+ (1+(int)(Math.random()*10)));
```



Sous probleme 5: ecrire des programme pour le restaurant yummy

programme 1:

Création du package `Yummy` :

Permet de regrouper tous les fichiers liés à l'entreprise dans un même espace logique.

Création d'une classe `Yummy` :

Le nom de la classe correspond au nom du fichier (`Yummy.java`) comme l'impose Java.

Définition de la méthode `main()` :

C'est le point d'entrée de l'application.

Utilisation de `System.out.println()` :

Permet d'afficher la phrase demandée dans la console.

```
<terminated> Yummy2 [Java Application] /Library/Java/JavaVirtualMachines/
Yummy prépare les meilleurs plats pour vos fêtes
```

programme 2:

Création du package `Yummy` :

Toujours dans le même package pour centraliser les fichiers.

Définition d'une nouvelle classe `Yummy2` :

Correspond au fichier `Yummy2.java`.

Définition du message dans une variable `String` :

Pratique pour réutiliser et centrer le message.

Calcul de la longueur totale :

On ajoute 4 caractères à la longueur du message pour tenir compte des `*` et des espaces autour.

Affichage de la bordure supérieure en boucle `for` :

On répète `*` autant de fois que nécessaire.

Affichage de la ligne contenant le message centré :

Avec des `*` à gauche et à droite.

Affichage de la bordure inférieure :

Même boucle que pour la bordure du haut.

```
<terminated> Yummy [Java Application] /Library/Java/JavaVirtualMachines/
*****
* Yummy prépare les meilleurs plats pour vos fêtes *
*****
```

programme 3:

Package `Yummy` : inchangé.

Classe `YummyFun` : nom correspondant au fichier `YummyFun.java`.

Stockage du message dans une variable `String`

Calcul de la longueur totale pour la bordure Comme précédemment, on ajoute 4 caractères (espaces + lettres à gauche/droite).

Boucle `for` pour afficher une ligne de `S` en haut et en bas.

Afficher le message entre deux `S`

SS
S Yummy, c'est fun au soleil S
SS

Programme en annexe

conclusion

Au cours de ce TP, j'ai pu découvrir différentes manières d'afficher des messages en Java, que ce soit dans la console ou à travers des interfaces graphiques simples avec JOptionPane.

J'ai rencontré quelques difficultés au départ avec les instructions permettant d'afficher une boîte de dialogue (popup), notamment concernant la syntaxe exacte à respecter et l'importation du bon package (`javax.swing`). Cela m'a permis de mieux comprendre la rigueur nécessaire en Java, où une petite erreur de syntaxe peut empêcher la compilation du programme.

Malgré ces petites difficultés, j'ai trouvé cette expérience intéressante et formatrice. Elle m'a permis de mieux comprendre la structure d'un programme Java, l'utilisation des chaînes de caractères, des boucles, ainsi que la notion de package pour organiser ses fichiers.

C'est une première approche simple mais concrète de la programmation orientée objet, et cela m'a motivé à approfondir mes connaissances dans ce langage.

Annexe

```
1 package yummy;
2
3 public class Yummy2 {
4     public static void main(String[] args) {
5         System.out.println("Yummy prépare les meilleurs plats pour vos fêtes");
6     }
7 }
8
```

```
1 package yummy;
2
3 public class YummyFun {
4     public static void main(String[] args) {
5         String message = "Yummy, c'est fun au soleil";
6
7         int longueur = message.length() + 4;
8
9         // Bordure du haut avec des S
10        for (int i = 0; i < longueur; i++) {
11            System.out.print("S");
12        }
13        System.out.println();
14
15        // Ligne avec le message entre 2 S
16        System.out.println("S " + message + " S");
17
18        // Bordure du bas avec des S
19        for (int i = 0; i < longueur; i++) {
20            System.out.print("S");
21        }
22        System.out.println();
23    }
24 }
25
26
```

```

1 package yummy;
2
3 public class Yummy {
4     public static void main(String[] args) {
5         String message = "Yummy prépare les meilleurs plats pour vos fêtes";
6
7         // longueur de la bordure = longueur du message + 4 espaces (2 de chaque côté)
8         int longueur = message.length() + 4;
9
10        // Afficher la bordure du haut
11        for (int i = 0; i < longueur; i++) {
12            System.out.print("*");
13        }
14        System.out.println();
15
16        // Afficher la ligne avec le message centré entre deux *
17        System.out.println("* " + message + " *");
18
19        // Afficher la bordure du bas
20        for (int i = 0; i < longueur; i++) {
21            System.out.print("*");
22        }
23        System.out.println();
24    }
25 }
26

```